

课程笔记

起零衍迹-Bytex-无径之径-Cairn AI 从渗透测试到通用问题的求解

腾讯安全众测 & Codex

2026-05-29



视频作者/频道：腾讯安全众测

发布日期：2026-05-01

视频时长：00:16:56

视 频 链 接：https://www.bilibili.com/video/BV1yQ9YBNuZ/?spm_id_from=333.1387.upload.video_card.click&vd_source=6457e444c35aa58365caef6c074dd705

目录

1 从信息收集到状态空间搜索	2
1.1 开场与问题设定	2
1.2 经典回答的边界：信息收集	2
1.3 未知状态空间	3
1.4 从动作到结构	3
1.5 通用问题求解的入口	4
1.6 本章小结	4
2 黑板图：把侦探破案变成系统状态	5
2.1 侦探黑板的类比	5
2.2 黑板架构的历史参照	6
2.3 三种图元素	6
2.4 图与路标	7
2.5 三类任务循环	8
2.6 OODA 对应关系	8
2.7 本章小结	8
3 从一道赛题看图如何长出来	9
3.1 示例的起点与终点	9
3.2 Bootstrap 的失败价值	10
3.3 Reason 产生新的 intent	12
3.4 Explore 执行并回写 fact	12
3.5 简单题与复杂题的图形差异	13
3.6 本章小结	14
4 Cairn 的工程骨架与间接协作	14
4.1 三块工程结构	14
4.2 图一致性与推理边界	15
4.3 Hint 注入与任务式指挥	15
4.4 预设角色与运行时任务	16
4.5 蚁群类比与间接协作	17
4.6 设计理念：Less is More	18
4.7 本章小结	18
5 赛场结果与 Less is More 的判断	19
5.1 赛场表现	19
5.2 系统成熟度的反差	19
5.3 讲者最后的判断	20
5.4 本章小结	21

6 评委问答：协议边界、成本、依赖与并发	21
6.1 外层协议的独立性：为什么不能只靠 Codex 或 Claude Code	21
6.2 Token 与金钱成本：架构代价已经暴露	22
6.3 依赖关系：由图态势推断，不靠系统预设	23
6.4 串行推理与并行执行：调度边界在哪里	23
6.5 瓶颈与未解决问题	24
6.6 本章小结	25
7 总结与延展：从渗透测试路径到通用求解骨架	26
7.1 讲者收束：成绩、风险与 Less is More	26
7.2 四个核心判断	26
7.3 什么时候适合这种架构	27
7.4 延展：从安全竞赛走向通用问题求解	27
7.5 本章小结	27

1 从信息收集到状态空间搜索

1.1 开场与问题设定

讲者来自 Bytex/BTX 战队，开场交代了几项背景：本届线上第四名、全场唯一 AK；第一届也参赛，成绩同样是第四名；上一届队名叫 ANTICKS。这些信息的作用是说明后面讨论的 Cairn AI 来自一次真实竞赛压力下的系统实验，光环叙事在这里没有教学价值。读者需要抓住的入口问题只有一个：如果把渗透测试从“经验技巧集合”提升到可建模对象，它到底在解决什么问题？

这场分享的标题里有“无径之径”。这个词很适合解释本章的方向：渗透测试看上去没有固定路径，因为每个目标、每个服务、每个错误配置都会把局面带向新的分支；它又有可被描述的结构，因为操作者总是从当前已知出发，面向一个明确目标推进。Cairn 的设计从这里开始：先找问题的形状，再决定系统怎样承载这个形状。

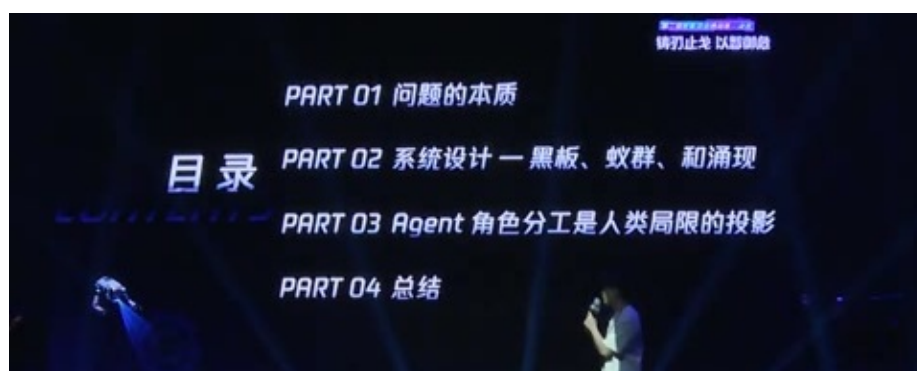


图 1: 演讲目录：从问题本质、系统设计到 Agent 分工与总结的整体结构。¹

1.2 经典回答的边界：信息收集

安全圈常说“渗透测试的本质是信息收集”。这个说法有用，因为渗透测试确实离不开发现：端口、服务、版本、页面、身份、配置、错误回显、权限边界，都会改变操作者对目标的理解。信息收

¹ 视频画面时间区间：00:00:27–00:00:36。

集像在黑暗房间里打开一盏一盏小灯，每盏灯照亮的范围有限，但它会暴露新的门、墙角和通道。

它的边界也很明显：信息收集描述的是动作层。动作层回答“现在做什么”，例如观察一个入口、确认一个现象、记录一个返回结果；它没有完整回答“系统怎样组织这些动作”，也没有回答“新发现怎样改变下一步”。如果只停在动作层，渗透测试会被写成工具清单或经验流程，读者很难看清后文为什么要引入图、路标和任务循环。

不要把信息收集写成全部

信息收集是状态空间中的移动方式。它提供新事实，改变后续判断，但它本身不等同于完整的求解结构。完整结构还包括起点、目标、可选路径、失败反馈、下一步意图，以及这些元素如何被系统保存和复用。

1.3 未知状态空间

“状态空间”可以先按集合来理解：一个系统在求解过程中可能处在许多状态里，每个状态都包含当前已知的事实、尚未验证的假设、可尝试的动作和目标距离。渗透测试面对的状态空间通常不可完全枚举。不可完全枚举不代表玄学，它只表示操作者无法在开始时列出所有可能情况。一个新页面可能引出新接口，一个错误信息可能暴露新路径，一个权限变化可能打开新的目标集合。

“有向搜索”里的“有向”来自目标。目标可以是拿到 shell、获得更高权限、取得全部 flag，也可以是确认某类风险是否存在。目标给搜索提供方向感，像山里的指南针：指南针不告诉人每一步踩哪块石头，但它让人知道偏离目标时需要调整。

用一条紧凑的公式，可以把本章的抽象写成：

$$\mathcal{P} : v_0 \rightsquigarrow V_T, \quad G = (V, E), \quad E = \{e_j : v_a \rightarrow v_b\}$$

- $\mathcal{P}$ ：一次求解过程，可理解为从起点朝目标推进的路径。
- v_0 ：起点状态，例如题目说明、入口地址、初始权限或当前已知信息。
- V_T ：目标状态集合，例如获得 shell、权限提升或拿到全部 flag；复杂问题常有多个达成条件。
- $G = (V, E)$ ：描述求解进展的图， V 是状态或事实节点， E 是由探索意图驱动的边。
- $e_j : v_a \rightarrow v_b$ ：第 j 条探索边，表示从一个已知状态尝试走向另一个状态。

这段公式的意义很朴素：渗透测试可以被看成在未知状态空间里的有向搜索。这里没有把安全问题包装成数学游戏；它给系统设计一个骨架：系统要能保存状态，要能表示边，要能在发现新事实后改变图。

1.4 从动作到结构

一旦引入状态空间，信息收集的位置就清楚了：它是让系统从一个状态移动到另一个状态的动作。比如当前状态只有一个入口地址，经过观察后发现登录页、接口路径或版本信息，状态就发生变化。这个变化可能带来新的动作候选：验证身份边界、检查配置暴露、关联题目提示。这里仍然保持概念层讲解，不展开任何可执行的攻击步骤。

从结构上看，完整渗透测试是一条或多条从起点通向目标集合的路径。路径更像探索者在地图上边走边补线：每次发现都会让地图多出节点或边，某些边会被证明走不通，某些边会连接到新的

²视频画面时间区间：00:00:39–00:01:46。



图 2: 渗透测试被抽象为近乎无限状态空间中的有向搜索。²

区域。失败也有结构价值，因为失败能把“这条路暂时无效”沉淀为已知，避免系统在同一处反复消耗。

本章核心定义

渗透测试在这里被定义为：从当前已知出发，面向目标终点的搜索过程。信息收集是这种搜索中的移动动作；状态空间承载所有可能局面；有向搜索说明目标已经给定，未知的是中间路径。

1.5 通用问题求解的入口

讲者把渗透测试、漏洞挖掘、数学证明放在同一类问题里，理由是它们共享一个结构：有明确起点，有达成标准，中间路径未知。数学证明的起点可能是一组公理和题设，终点是命题成立；漏洞挖掘的起点可能是代码、接口和运行环境，终点是确认某个可利用风险或排除某类风险。它们的领域知识不同，求解形状却相似。

这就是“通用问题求解引擎”这个说法的合理边界。它不能被夸大成 Cairn 已经具备通用智能，也不能把所有问题压扁成同一种算法。更准确的说法是：当一个问题满足“目标明确、路径未知、探索结果可回写、失败信息有价值”这些条件时，图上的有向搜索提供了可复用的系统骨架。

读者可能忽略的一层

很多工程系统失败在“目标”和“过程”被混在一起。目标是达成标准，例如拿到全部 flag；过程是当前选择的路径，例如先观察哪个入口。Cairn 的抽象价值在于把路径选择交给可更新的图，而让目标始终作为搜索方向存在。

1.6 本章小结

本章要带走的句子很短：Cairn 的核心对象从单个渗透技巧上移到搜索结构本身。信息收集仍然重要，但它被放回“状态空间移动”这个位置；渗透测试从经验动作串上升为从已知起点通向目标集合的有向搜索。后文引入黑板图，就是为了回答这个搜索结构怎样被表达、推进和复用。

³视频画面时间区间：00:01:48-00:02:10。



图 3: 从渗透测试、漏洞挖掘、数学证明到通用问题求解引擎的共同结构。³

2 黑板图：把侦探破案变成系统状态

2.1 侦探黑板的类比

讲者用侦探破案来解释 Cairn 的黑板图。想象一群侦探围着同一块黑板：案件资料是起点，找出凶手是终点，板书上的确定信息是已经确认的事实，粉笔画出的问号代表下一步想查的方向，便利贴上的提醒来自主侦探或外部线索。有人在黑板上写下怀疑，有人离开房间去验证，回来后把结果写回黑板。

这个类比解决了两个理解难点。第一，所有参与者共享同一个上下文；没有共享黑板，每个人都会拿着自己的小纸条重复猜测。第二，行动和记录形成闭环；只行动不记录，系统无法积累；只记录不行动，图会变成静态档案。黑板图的意义就在于让“知道了什么、打算做什么、做完后得到什么”落到同一个公共状态里。



图 4: 侦探黑板类比：fact 节点、intent 边与外部 hint。⁴

⁴ 视频画面时间区间：00:02:13-00:03:04。

2.2 黑板架构的历史参照

黑板架构 (Blackboard Architecture) 是一个旧概念。20 世纪 70 年代的语音识别研究中，多个独立知识源会围绕一个共享黑板协作：某个知识源识别音素，另一个知识源尝试词汇匹配，还有知识源处理句法约束；它们不必两两直接通信，只需要读取和更新公共状态。

这个历史参照不需要展开成考据。它的教学价值在于提醒读者：当问题需要多种知识片段逐步拼合时，共享状态比点对点消息更稳。侦探破案、语音识别、复杂问题求解都遇到同一类压力：局部发现需要被其他过程看见，后续动作需要建立在全局状态之上。

黑板架构的直觉

黑板架构像一个公共白板会议室。参与者可以有不同能力，协作中心落在所有人都能看见、能更新、能追溯的公共状态上，私聊记录只会让上下文分散。Cairn 把这个公共状态做成图，所以以后文统一称为黑板图。

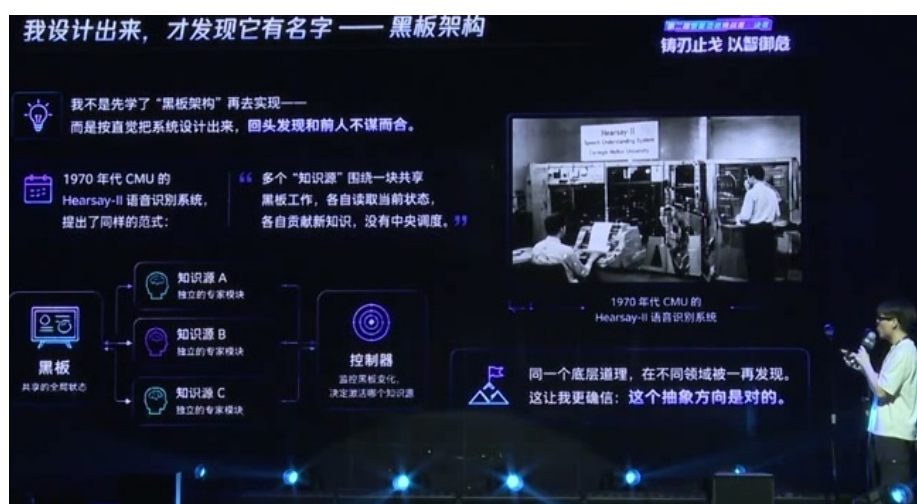


图 5: 黑板架构参考：多个知识源围绕共享全局状态协作。⁵

2.3 三种图元素

Cairn 的黑板图里有三类关键元素：fact、intent、hint。

- **fact** 是已经确认的事实。对应侦探黑板上写实的板书，例如“某个线索已验证”“某个入口存在”“某次尝试得到某个结果”。它的关键词是确认。
- **intent** 是下一步要尝试的动作或探索声明。对应黑板上画出的问号和箭头，例如“去查这个方向”“验证这两个事实是否能组合”。它的关键词是意图。
- **hint** 是来自图外的高维提示。它可能来自人类、题目本身或外部上下文，对图内推理提供参考。它的关键词是外部参考。

三者的分工很重要。一个 **fact** 像已经钉在板上的照片，不能随便当猜测使用；一个 **intent** 像从照片拉出的红线，说明下一步准备检查什么；一个 **hint** 像贴在板边的便签，提醒侦探注意某种方向，但它不直接替代图内推理。

⁵ 视频画面时间区间：00:03:05–00:03:34。

fact / intent / hint

fact 记录确认事实，**intent** 记录探索意图，**hint** 注入外部提示。三者共同让黑板图同时保存上下文、方向感和人为输入，避免把“已经知道的事”和“准备尝试的事”混在同一个文本堆里。

2.4 图与路标

在这套抽象下，黑板图从起点出发，逐步连接到目标。讲者提到 **turn** 这个命名，可理解为路径推进中的路标或转向点：走到这里，系统已经知道了一些东西，也产生了下一步方向。具体英文术语仍应由画面代理复核，写作上暂按 **turn** 处理。

把黑板图写成时间变化的形式，可以看到它会随任务推进增量生成：

$$G_t = (V_t, E_t, H_t), \quad V_t = \{f_1, \dots, f_m\}, \quad E_t = \{e_1, \dots, e_n\}, \quad h_k \in H_t$$

- G_t : 第 t 个路标时刻的黑板图。
- V_t : 当前图中的节点集合，这里主要承载 **fact**。
- E_t : 当前图中的边集合，常由 **intent** 驱动。
- H_t : 当前可参考的提示集合，包含人类或题目注入的 **hint**。
- f_i : 第 i 个已确认事实。
- e_j : 第 j 条探索边，可能已经执行，也可能仍指向未知结果。
- h_k : 第 k 个外部提示。

这个公式没有把系统变复杂；它只是把“图会长大”这件事说清楚。每个时刻的 G_t 都是当下态势，下一次行动会根据它生成；行动结果回写后，系统进入新的 G_{t+1} 。



图 6: Cairn 黑板: fact、intent、hint 与从 origin 到 goal 的图路径。⁶

⁶视频画面时间区间：00:03:35–00:04:14。

2.5 三类任务循环

在黑板图之上，Cairn 把 agent 的工作压缩成三类任务：`bootstrap`、`reason`、`explore`。字幕里出现的“put strap”应按语义规范为 `bootstrap`。

`bootstrap` 是初始尝试。系统拿到一道渗透测试场景或通用问题后，先尝试直接解决；大多数复杂场景很难一次成功，但失败会留下有价值的事实，例如尝试过什么、哪里走不通、发现了什么新信息。

`reason` 是读取全图后的推理。它避开局部一句话式判断，转而关注整个黑板图的态势：哪些事实已经确认，哪些意图还没执行，哪些提示可能影响方向。它的输出通常是新的 `intent`。

`explore` 是执行意图并回写结果。它拿到 `reason` 产生的路径，去验证、观察或推进，然后把结论写回图。这样一来，下一轮 `reason` 看到的是包含新增事实或失败记录的新图。

$$T \in \{\text{bootstrap}, \text{reason}, \text{explore}\}, \quad \Delta G = G_{t+1} - G_t$$

- T : worker 本轮承担的任务类型。
- `bootstrap`: 初始试解并沉淀发现。
- `reason`: 读取全图并生成下一步意图。
- `explore`: 执行意图并把结果回写为图变化。
- ΔG : 一次任务后图的变化，可能是新增事实、新增意图，也可能是失败记录。

失败也是事实来源

在黑板图里，失败尝试不只是“没有成功”。只要失败被结构化记录，它就会减少重复探索，暴露隐藏约束，并为下一轮推理提供反证材料。没有回写的失败只会变成消耗。

2.6 OODA 对应关系

讲者把这套循环对应到 OODA: Observe、Orient、Decide、Act，也就是观察、分析、决定、执行。这里的 OODA 只作为解释支架使用。观察对应读取新事实，分析对应理解图的态势，决定对应产生下一步 `intent`，执行对应 `explore` 的行动和回写。

更直观地说，黑板图承担“看见”和“记住”的功能，`reason` 承担“想清楚下一步”的功能，`explore` 承担“把想法放到环境中验证”的功能，`bootstrap` 则负责在一开始制造第一批可用事实。这个循环的关键不在四个英文词，而在图的闭环：观察必须进入图，决定必须来自图，执行结果还要回到图。

2.7 本章小结

本章把上一章的“有向搜索”落到了一个可保存、可更新、可调度的公共状态里。黑板图用 `fact` 保存确认事实，用 `intent` 保存下一步探索方向，用 `hint` 接收外部高维提示；`bootstrap`、`reason`、`explore` 让图在失败、推理和执行之间持续生长。读者到这里应当形成一个清晰图景：Cairn 不靠预写一套渗透流程推进，下一步行动从当前黑板图的态势中产生。

⁷ 视频画面时间区间：00:04:16–00:05:04。



图 7: Agent 工作循环：Bootstrap、Reason、Explore 与 OODA 映射。⁷

3 从一道赛题看图如何长出来

上一章已经把 Cairn AI 的黑板图、fact、intent、hint 以及 bootstrap / reason / explore 三类任务放在同一套语言里。本章只做一件事：把这些抽象词放回一条真实赛题轨迹中，观察图从空白的起点逐步长出路径。

这里的关键观察很朴素：系统没有预置一套渗透测试流程清单，也没有把“该怎么做”写成硬编码逻辑。worker 能看到的只有当前黑板图，以及本轮被分配到的三类任务之一。于是每一步推进都必须从图的当前态势中产生，像侦探黑板上的粉笔线一样，先留下尝试痕迹，再让后续判断接住这些痕迹。



图 8: 赛题生命周期图强调 Agent 每次只看到当前图与三类任务指令，并且没有硬编码的渗透测试流程。⁸

3.1 示例的起点与终点

这道线上真实题目的起点很明确：题目给出的全部信息，加上可访问的 IP 入口。用前文符号记作 v_0 ，它指向“系统一开始确实知道什么”的集合，不需要被理解成某个神秘的初始智能状态。题

⁸ 视频画面时间区间：00:05:10–00:05:26。

目描述、入口地址、可见约束、题目自带提示，都属于这个起点的一部分。

终点同样明确：获取全部 flag。更严谨地说，终点可以写成目标状态集合 V_T ，因为复杂赛题里可能存在多个 flag、多个子目标、多个验收条件。当前示例中，字幕强调的是“获取全部 flag”，所以读者可以把 V_T 理解为所有目标条件被满足的状态。

起点和终点决定了搜索方向

在这一章里，有向搜索的“有向”来自目标状态集合 V_T 。系统面对的是大量未知中间路径，方向感来自终点约束；它不会因为中间发现很多新事实，就忘记最终要抵达哪里。

把起点和终点钉住之后，黑板图的第一层含义就出现了：它首先是一种运行时状态，事后复盘图只是在运行结束后的投影。每个新增事实都会改变图，图改变之后，下一轮推理读取到的上下文也随之改变。

3.2 Bootstrap 的失败价值

bootstrap 的任务看上去很直接：拿到题目之后，先尝试直接解决。大多数情况下，这一步不会马上成功。讲者在这里强调的重点是，直接解决失败不代表这一轮没有产出；系统会把尝试过程和新发现写回图里。

这正是黑板图区别于普通日志的地方。普通日志像流水账，主要记录“刚才发生了什么”；黑板图更像侦探白板，记录的是后续推理可以继续利用的结构化线索。失败尝试如果发现了入口响应、认证边界、页面线索、服务特征、异常反馈，它们都可以沉淀为新的 **fact**。这些事实不需要立刻完成目标，也能改变下一轮 **reason** 的判断。

失败是图增长的燃料

在 Cairn AI 的语境里，一次失败的 **bootstrap** 至少可能贡献两类东西：第一类是已经确认的 **fact**，也就是尝试后确实观察到的事实；第二类是被排除或被降权的方向，它让后续 **reason** 少走同样的路。失败的价值来自可回写、可引用、可触发下一轮判断。

可以把这一轮变化压缩成一个很小的关系。先用朴素说法理解：worker 尝试直接求解，失败后把“试过什么”和“看到了什么”写回图，于是图从 G_t 变成 G_{t+1} 。

$$G_{t+1} = G_t \cup \Delta G_t, \quad \Delta G_t = \{f_{\text{try}}, f_{\text{obs}}\}$$

G_t ：第 t 轮任务开始前的黑板图。

G_{t+1} ：本轮写回之后的黑板图。

ΔG_t ：本轮任务给图带来的增量。

f_{try} ：关于“尝试过什么”的事实，例如某个方向已经被尝试。

f_{obs} ：关于“观察到什么”的事实，例如入口、响应、约束或异常现象。

这个公式没有把安全操作细节写出来，因为本章关心的是求解机制。真正重要的是： ΔG_t 让失败从一次消耗变成可复用上下文。后续任务读取的是题面加上第一轮探索痕迹之后的图。

⁹视频画面时间区间：00:05:28–00:05:43。

¹⁰视频画面时间区间：00:05:45–00:05:58。

¹¹视频画面时间区间：00:06:17–00:06:25。

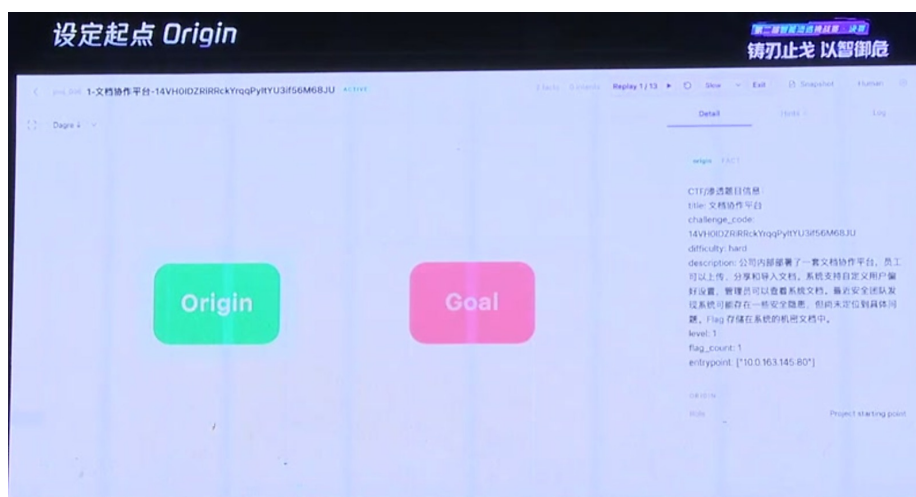


图 9: 真实 CTF 题目的初始图状态: 系统先把题目信息与 IP 入口抽象成 Origin, 并把拿到全部 flag 抽象成 Goal。⁹



图 10: Bootstrap 未能直接到达 Goal, 但把尝试中的发现写回为新的 Fact, 给后续 Reason 提供依据。¹⁰



图 11: Worker 沿着图上生成的 intent 继续执行, 最终连接到 Goal, 形成完整的解题路径。¹¹

3.3 Reason 产生新的 intent

当 bootstrap 写回新事实之后，系统进入 reason 阶段。reason 的职责集中在读图和判断下一步方向上，执行动作交给后续 explore。它读取的是当前完整黑板图：起点 v_0 、已有 fact、未完成 intent、外部 hint，以及这些元素之间的连接关系。

这一点很容易被低估。若把图想成地图，reason 做的事情接近“站在最新地图前决定下一条路标画在哪里”。它的输出来自当前事实中可以继续推进的方向。字幕里说“推理出了一个新的意图”，落到黑板图上，就是新增一条待探索的 intent 边。

$$e_{t+1} : f_a \xrightarrow{\text{intent}} v_?$$

e_{t+1} ：第 $t+1$ 轮生成的探索边。

f_a ：支撑该意图的上游事实，可能是一个事实，也可能是一组事实的汇合点。

$\xrightarrow{\text{intent}}$ ：由 intent 驱动的有向探索关系。

$v_?$ ：尚未确认的目标节点，等待 explore 执行后才能变成确定事实或被排除。

这条边的含义很具体：系统认为“从 f_a 出发，值得去尝试某个方向”。在真实运行中，这个方向可能指向认证、入口验证、约束检查、路径确认等概念层面的任务。为了保持安全边界，本节只讨论图上关系，不展开任何可直接复现的攻击步骤。

不要把 intent 读成固定脚本

intent 是下一步探索声明，来源于当前图的态势。它不等同于预先写死的渗透测试脚本，也不等同于一条具体工具命令。若图里的事实变化，同一个起点可能生成不同的 intent；若事实不足，reason 也可能先生成补充信息的意图。

3.4 Explore 执行并回写 fact

explore 接住 reason 画出的边。worker 执行这条 intent，得到新的观察结果，再把结果写回黑板图。于是循环又回到上一层：图变化触发新的推理，新的推理产生下一条边，下一条边继续被执行。

可以把这一步写成：

$$W(e_{t+1}) \Rightarrow f_b, \quad G_{t+2} = G_{t+1} \cup \{f_b\}$$

$W(e_{t+1})$ ：worker 执行第 $t+1$ 条探索边。

$\Rightarrow$ ：执行后产生可写回的结果。

f_b ：执行后确认的新事实；它可能推动目标前进，也可能证明某条路暂时走不通。

G_{t+2} ：新事实写回后的黑板图。

这一轮结束后， f_b 不再属于某个 worker 的私有记忆。它进入共享黑板图，成为后续 reason 的公共上下文。这个设计让协作不依赖 worker 之间的私聊，也减少了“某个执行者知道线索，调度层却不知道”的断裂。

在字幕描述的简单题里，若干轮“生成意图、执行、写回事实”之后，某一条 intent 边已经通向终点，系统获得 flag。这里的“通向终点”不需要被神秘化：它只表示新事实满足了目标集合 V_T 中的验收条件。换句话说，目标达成需要图上出现足以证明目标达成的事实，口头宣布抵达不够。

3.5 简单题与复杂题的图形差异

讲者随后给出一个对照：前一道题较简单，图更像一条主路径；复杂题的真实图会出现更多节点、更多边、更多分叉和汇合。这个差异很关键，因为它说明 Cairn AI 的抽象并不假设所有问题都能线性解决。

简单题的图可以想成一条山路：起点清楚，几次转弯之后抵达终点。复杂题更像城市路网：有些路只负责确认前提，有些路会汇合成一个更强的事实，有些路走到一半被放弃，还有些边可以并行探索。黑板图的优势在复杂图里更明显，因为所有事实、意图和回写结果都沉淀到同一个公共状态中。

从数学直觉看，简单题的主干可以近似写成一条链：

$$v_0 \rightarrow f_1 \rightarrow f_2 \rightarrow \cdots \rightarrow V_T$$

v_0 ：题目全部信息和 IP 入口构成的起点。

$f_1, f_2, \dots$ ：沿路逐步确认的事实。

V_T ：全部 flag 被获取时满足的目标状态集合。

$\rightarrow$ ：由 **intent** 和 **explore** 推动的状态推进关系。

复杂题的图会出现分叉与汇合。分叉表示同一批事实可能支撑多条探索边；汇合表示多个上游事实共同支撑一个后续意图。此时图的价值不在于画得漂亮，而在于让系统清楚知道：哪些事实已经确认，哪些意图等待执行，哪些尝试已经失败，哪些目标条件已经接近满足。



图 12: 更复杂案例中的图开始出现分叉、汇合和多条候选路径，说明图增长会随题目复杂度扩展。¹²

这一节的运行轨迹

Section 3 的最小闭环是：从 v_0 出发，**bootstrap** 尝试并回写失败中的发现，**reason** 读取全图生成新的 **intent**，**explore** 执行意图并写回 **fact**，图的增量 ΔG 再触发下一轮判断。循环持续到某条路径满足 V_T 。

¹²视频画面时间区间：00:06:25–00:06:43。

3.6 本章小结

这一章展示的是黑板图如何在真实赛题中生长。起点来自题目全部信息和 IP 入口，终点来自全部 flag 的达成条件；bootstrap 的失败提供第一批可复用事实；reason 把当前图态势转化为新的 intent；explore 执行这些意图，再把结果写回为新的 fact。简单题可能表现为一条清晰主路径，复杂题会长出分叉、汇合和多轮回写。

读者需要带走的是一种运行机制，具体赛题技巧只在这里承担例证作用：Cairn AI 把问题推进过程沉淀到共享黑板图里，让下一步行动由图的当前态势产生。只要失败信息有价值、探索结果能回写、目标状态足够明确，这套循环就能把抽象的有向搜索落到一条可观察、可审计、可继续推进的求解轨迹上。

4 Cairn 的工程骨架与间接协作

上一章已经把 Cairn 的运行轨迹放到一张黑板图上观察：新的 fact 写入图，新的 intent 从图中长出，worker 再沿着某条意图边去执行。第 4 节需要回答更工程化的问题：这张图由谁保存，谁负责读写和派发，真正执行动作的 agent 又被装在哪里。

4.1 三块工程结构

讲者把 Cairn 的工程结构概括成三块。第一块是 **Cairn Server**，它更接近协议层或黑板层，维护图中核心数据结构的一致性。第二块是读写图的调度层，合同中暂称为 **graph I/O dispatcher**，负责把读图、写图和容器调用连接起来。第三块是 **worker containers**，容器里承载 Codex、Claude Code 或讲者自己实现的轻量 agent。

这里有一个必须克制处理的术语问题：ASR 把 Cairn Server 识别成 **current server**，把 dispatcher 识别成 **DESPTURE**，把 Claude Code / Codex 一类执行器识别成近似音。正文采用大纲合同中的规范名；精确英文拼写应由图像代理在架构页面画面中复核。



图 13: Cairn 的工程骨架：Cairn Server 维护 Facts、Intents、Hints，Dispatcher 负责调度与容器管理，容器内运行 Claude Code、Codex 等执行器。¹³

Cairn Server 的职责边界

这里先抓住一条结构线索：Cairn Server 保存公共状态，推理边界会在下一小节展开。

如果把黑板图写成 $G = (V, E)$ ，可以用一个很轻的更新式描述 dispatcher 的职责：

$$G_{t+1} = \text{merge}(G_t, \Delta G_w)$$

G_t ：第 t 轮时 Cairn Server 保存的黑板图。

G_{t+1} ：合并 worker 回写结果后的新图。

ΔG_w ：某个 worker 本轮执行后产生的图变化，可能是新增 **fact**、新增 **intent**，也可能是对旧节点状态的补充说明。

merge：dispatcher 与 server 共同完成的合并动作，核心要求是保持图的一致性，避免同一轮写回把公共状态冲散。

这个公式的作用有限，只帮助读者抓住“图是公共状态，worker 只提交变化量”。公式只说明状态如何回写；真正推动下一步行动的是图变化触发的 **reason**，以及 worker 对新 **intent** 的执行。

4.2 图一致性与推理边界

Cairn Server 的职责边界需要说清楚。讲者明确强调它“只管图的一致性”，不做任何推理。推理发生在任务层，由 worker 在读取当前图之后完成。这个设计的工程意义很直接：公共状态有统一入口，推理能力可以外包给不同执行器，系统协议仍然保持稳定。

这也解释了为什么 Cairn 可以约束 worker。worker 看到的上下文来自黑板图，接到的任务来自三类任务命令之一，写回也落到图结构中。无论底层执行器是 Codex、Claude Code，还是轻量自研 agent，它们都被夹在同一套图读写协议里。换成操作系统类比，Cairn Server 像文件系统的元数据层，dispatcher 像 I/O 调度器，worker 像运行在隔离进程里的程序；程序可以很强，但文件系统要保证写入仍然落在可恢复、可审计的结构里。

不要把状态层误读成推理层

不能把 Cairn Server 理解成“推理服务器”。一旦把推理职责塞回 server，系统会把状态一致性、任务生成和执行决策混在同一层，后续很难判断错误来自图写坏、推理误判，还是执行器失败。讲者选择把 server 压到状态层，正是为了让责任边界变硬。

4.3 Hint 注入与任务式指挥

人类和题目给出的高维输入通过 **hint** 注入图。这里的 **hint** 并非普通 prompt 的同义词，它更像贴在案件黑板旁边的便签：便签提供方向、约束或背景，但不亲自参与图内推理，也不替 worker 写出每一步操作。

讲者用“任务式指挥”来解释这层关系。mission command 的核心是：上层表达意图，下层根据现场态势自主决策。放到 Cairn 中，人类给出目标、提示或约束，worker 读取图中的当前态势，再判断本轮应该如何完成任务。人类不下发逐条细化命令，系统也不依赖人工把渗透测试流程拆成固定脚本。

¹³ 视频画面时间区间：00:06:47–00:07:44。

任务式指挥的系统含义

任务式指挥在 Cairn 中的含义：人类给出 hint 或目标意图，图保存当前态势，worker 根据这张图生成和执行本轮动作。控制力来自公共状态和任务协议，细节决策留给执行器在现场完成。

这个设计给读者补上一点常被忽略的工程常识：prompt 微操很容易制造“看似可控”的错觉。指令写得越细，系统越像复读人工流程；一旦环境出现新事实，固定流程需要人工不断补丁。Cairn 的选择是把人类输入收敛成 hint，把动态部分交给图态势和 worker 的本轮任务。

4.4 预设角色与运行时任务

传统多 agent 方案常把分工写在设计时，例如信息收集 agent、漏洞利用 agent、报告 agent。这样的分工容易解释，也容易画架构图。代价在于角色边界会提前切开上下文：一个角色拿到的信息、过滤掉的信息、翻译过的信息，都可能影响下一个角色的判断。

Cairn 采用另一种分工来源：任务在运行时由图的当前态势产生。新的 fact 写入图，态势变化触发新一轮 reason，reason 产生新的 intent，随后某个 worker 执行 explore。这里的分工仍然存在，只是分工不由人类在架构设计阶段钉死，而由图在运行过程中生成。

可以把它理解成厨房出单系统。厨师不会被贴上“永远只切菜”或“永远只煎炒”的标签。订单来了，系统根据当前菜品状态、缺什么工序、哪个台面空闲，把下一步任务交出去。Cairn 的 worker 也类似：本轮可能承担 reason，下一轮可能承担 explore，另一个 worker 同时也可能承担推理或探索。worker 的身份轻，任务的上下文重。

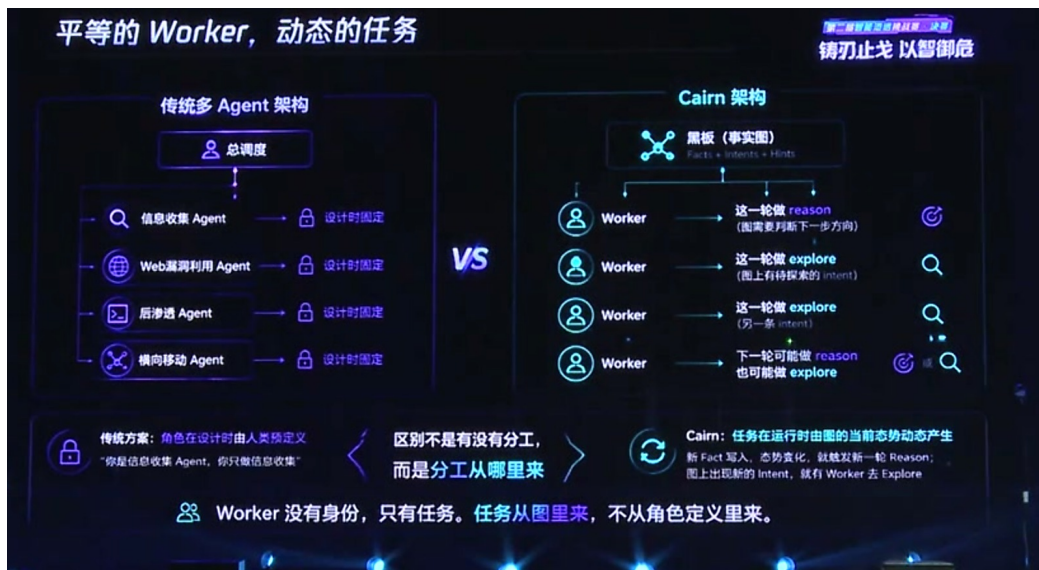


图 14: 传统多 Agent 架构与 Cairn 架构的对照：左侧角色在设计时固定，右侧 Worker 平等，任务由运行时图态势动态产生。¹⁴

¹⁴ 视频画面时间区间：00:07:46–00:08:56。

运行时任务分配

运行时任务分配：worker 没有固定身份，只有本轮任务。任务来自图的当前态势： ΔG 改变了黑板图，reason 读图后产生新 **intent**，worker 再按该 **intent** 执行。系统的分工来源从“人类预设角色”移动到“图状态驱动任务”。

4.5 蚁群类比与间接协作

讲者随后引入蚁群觅食的类比。蚂蚁之间不需要逐个开会沟通，却能通过路径上的信息素浓度形成群体路线。更短、更有效的路径会留下更强的环境痕迹，后续个体沿着这些痕迹行动，群体行为因此涌现出来。

Cairn 借用的是这个类比中的“间接协作”，技术上可称为 stigmergic coordination。worker 之间不做点对点通信，它们通过黑板图协作：一个 worker 写入新 **fact**，另一个 worker 在下一轮读图时看到它；一个 worker 写入新 **intent**，后续 worker 沿着这条意图边继续探索。图就是共享信息素环境。

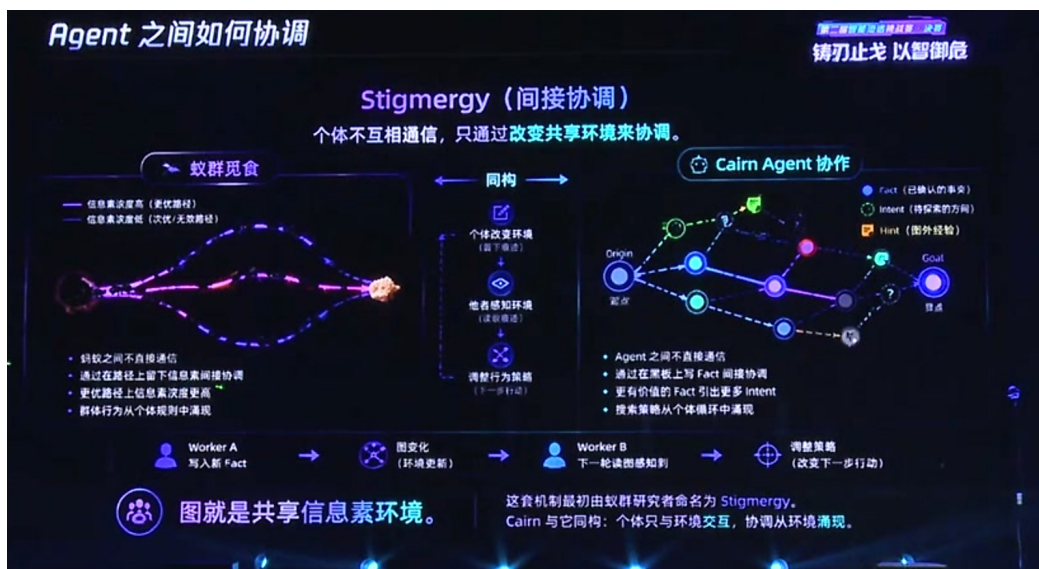


图 15: 蚁群信息素类比解释了间接协作：个体通过改变共享环境影响后续路径选择，Cairn 中则通过写入 Fact 与 Intent 推动协作。¹⁵

涌现的朴素含义

“涌现”在这里有一个朴素含义：系统整体表现超过单个 worker 的显式规则清单。每个 worker 只读图、执行当前任务、写回事实或意图；多轮之后，渗透测试的推进路线和搜索策略从这些局部循环中累积出来。

这个类比也有边界。蚁群信息素通常是连续浓度，黑板图中的 **fact** 和 **intent** 更像结构化记录。它们的共同点在协作方式：个体把痕迹留在环境里，后续个体通过环境痕迹被引导。Cairn 要借的正是这点，没必要把它扩写成完整的蚁群优化算法。

¹⁵ 视频画面时间区间：00:08:57–00:09:58。

4.6 设计理念：Less is More

讲者把设计理念收束为 *Less is More*。在他的描述中，Cairn 没有内置渗透工具，没有把 MCP、RAG、固定流程定义或预定义 agent 流程当作核心层。这个说法容易被误读成“少用技术栈就会更强”，实际要抓住的是更尖锐的工程判断：额外层数会带来上下文孤岛、翻译损耗和责任边界漂移。

复杂架构常见的诱惑是不断加层：缺能力就加一个模块，遇到新场景就加一个子 agent，流程不稳定就再包一层调度。这样做短期直观，长期会让系统在问题外面堆解法。Cairn 的路径是向内压缩，追问渗透测试的最小状态表达是什么，通用问题求解的共同骨架是什么，然后把系统层数压到图、任务、worker 的循环上。



图 16: Less Is More 设计页列出 0 个渗透工具 MCP、0 个安全领域 RAG、0 个渗透流程定义、0 种预定义 Agent 角色，核心在于从问题内部寻找抽象。¹⁶

Less is More 的风险

Less is More 不是廉价简化。少一层需要更深的抽象判断：被去掉的层到底是噪声、重复协议、人工流程投影，还是当前系统真正需要的能力边界。删错层会让系统失去约束；删对层才可能让状态空间的表达更接近问题本身。

这句话也解释了 Cairn 的雄心和风险。雄心在于它试图用最小图表达覆盖渗透测试、漏洞挖掘、数学证明等有起点、有目标、有未知路径的问题。风险也在这里：如果图结构没有真正抓住问题本质，少层架构会很快暴露能力缺口，后续补救空间反而变窄。

4.7 本章小结

第 4 节的核心可以压缩成一句话：状态由黑板图统一保存，任务由图的当前态势生成，协作通过共享状态发生。Cairn Server 保证图一致性，dispatcher 连接图读写和容器调用，worker containers 承载具体执行器。人类通过 hint 注入方向，worker 按任务式指挥在现场自主决策。蚁群类比帮助读者看见间接协作的机制：worker 之间不互相喊话，它们把事实和意图写到环境里，让后续行为被公共图牵引。

¹⁶ 视频画面时间区间：00:10:02-00:10:58。

5 赛场结果与 Less is More 的判断

第 5 节从工程主张转向结果检验。讲者没有只展示架构图，也给出赛场表现和系统成熟度的反差信息：线上第四名、最终唯一 AK，同时赛前没有完整测试，比赛当天凌晨四点才第一次调通。这些事实必须一起读。

5.1 赛场表现

讲者给出的结果有三层。第一，队伍线上排名第四。第二，开局配置失误导致前期抢分受损。第三，最终成为全场唯一 AK 所有题目的队伍。对一个自动化或半自动化渗透测试系统来说，唯一 AK 是强信号，说明这套图中心机制在真实比赛约束下产生了有效搜索。



图 17: 讲者把赛场结果和三条设计判断压缩到同一页：有向搜索、完整上下文中的 worker、以及在问题内部寻找最小表达。¹⁷

赛场结果的合理解读

赛场结果的合理解读：唯一 AK 说明 Cairn 的方向有真实信号，尤其说明图驱动搜索循环在至少一个高压任务集中跑出了结果。它还不能自动推出工程成熟、稳定可复现、跨场景泛化已经被充分验证。

“信号”这个词在这里很重要。一次比赛结果像一次高质量压力测试，能过滤掉很多纯概念系统；它也只是个样本。系统要从赛场原型走向稳定工程，还需要更多题型、更多失败案例、更多对照实验和更清楚的成本边界。

5.2 系统成熟度的反差

讲者紧接着补充了另一组事实：这套系统赛前没有经过任何测试，是比赛当天凌晨四点才第一次调通。这个细节不应被包装成浪漫叙事。它是工程风险证据，说明系统当时很可能仍处在原型阶

¹⁷ 视频画面时间区间：00:11:03-00:11:34。

段，稳定性、可恢复性、日志可追踪性、失败处理和长期运行行为都没有经过充分论证。

这里的风险可以分成三类。第一是部署风险，开局配置失误已经造成前期抢分损失。第二是验证风险，缺少赛前完整测试意味着很多失败路径没有被提前暴露。第三是复现风险，凌晨四点首次调通的系统跑出好成绩，可能同时依赖赛题结构、执行器状态、人工介入节奏和当场环境。

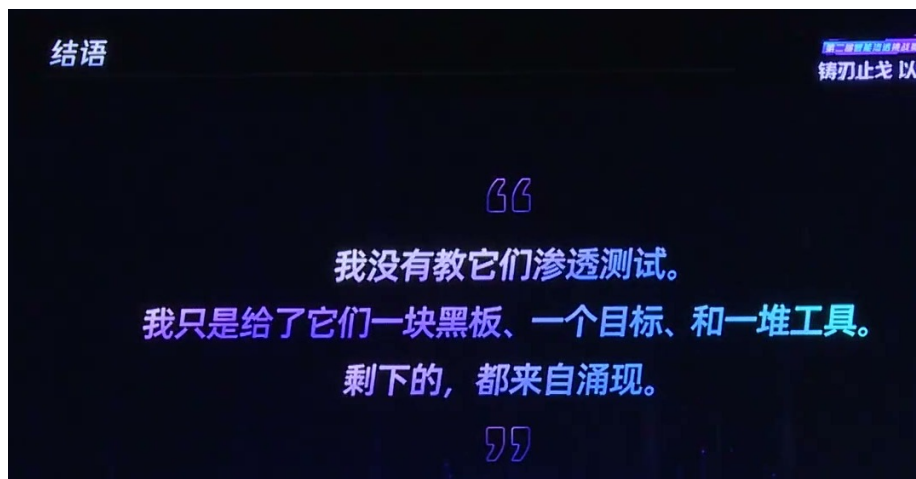


图 18: 结尾页把 Less is More 落到黑板、目标与工具的组合上：系统给出共享状态和目标，剩余路径从图中涌现。¹⁸

成绩与风险必须一起读

这组事实必须保留原本的张力：比赛成功是方向信号，赛前未完整测试和凌晨四点首次调通是工程风险证据。把后者写成“临场发挥强”会误导读者低估系统可靠性问题。

读者还应意识到一个“自己可能不知道自己不知道”的点：AI agent 系统的失败经常表现为状态慢慢偏移，未必立刻出现单点崩溃。一次错误写回、一次上下文丢失、一个意图边命名不清，短时间可能不报错，多轮之后会把搜索引向错误区域。没有测试的系统尤其难发现这种慢性失真。

5.3 讲者最后的判断

讲者把赛场结果拉回一个更朴素的判断：去掉一个东西，需要比增加一层更深的理解。这个判断承接第 4 节的 Less is More，也被赛场事实拉紧了。一边是唯一 AK 带来的有效信号，另一边是赛前未完整测试和凌晨四点首次调通暴露出的工程风险；少层架构到底抓住了问题本体，还是暂时跑通了一个高压样本，需要后续任务继续验证。

这个判断可以落到两条工程准则上。第一，只有当一个模块能改变图表达、任务生成或执行反馈时，它才值得进入核心架构。第二，新增层数需要证明它降低了状态复杂度、减少了上下文损耗，或提高了失败可诊断性；如果只是把人工流程外包给另一个 agent，它可能只是在复制人的局限。

Less is More 的工程版本

Less is More 的工程版本可以这样理解：少层数不是目标本身，少错层数才是目标。Cairn 试图保留三件事：公共图、运行时任务、可替换 worker。其他能力可以作为执行器能力或外部资源出现，但不要轻易变成新的核心控制层。

¹⁸ 视频画面时间区间：00:11:32-00:11:34。

这个判断也给第 6 节的问答埋下伏笔。评委后续追问外层协议、Token 成本、依赖关系和并发边界，实际上都在检验 Less is More 是否只是口号，还是已经变成可审查的工程约束。

5.4 本章小结

第 5 节的基调应保持克制。线上第四和唯一 AK 说明 Cairn 的设计方向值得重视，尤其是图中心搜索、运行时任务分配和间接协作在赛场中产生了效果。开局配置失误、赛前没有完整测试、凌晨四点才首次调通，同样说明这套系统的工程稳定性还需要被证明。Less is More 在这里不是胜利宣言，它更像一条待继续验证的工程假设：当系统层数减少到黑板图、任务循环和 worker 执行时，是否真的能更稳地逼近复杂问题的本质。

6 评委问答：协议边界、成本、依赖与并发

前面的章节给出了 Cairn AI 的正面叙述：把渗透测试抽象成黑板图上的有向搜索，用 `fact`、`intent`、`hint` 和三类 worker 任务来推进状态。问答环节的价值在于把这套叙述拉回工程审查现场。评委追问的焦点很直接：如果底层执行已经交给 Codex 或 Claude Code，外层协议还有什么必要；如果每轮都要带着图上下文调用模型，成本会不会失控；如果任务之间存在依赖，系统如何同时处理顺序约束与并发分发。

6.1 外层协议的独立性：为什么不能只靠 Codex 或 Claude Code

评委首先要求讲者翻到调用 Codex 和 Claude Code 的页面，并确认一个关键事实：Cairn 的上层会先构建一个较小的 `intent`，再把这个 `intent` 交给 Codex 或 Claude Code 一类执行器。这个回答说明了系统边界：Codex 和 Claude Code 在这里承担执行器或 worker runtime 的位置，Cairn 的黑板图与调度协议位于它们外侧。

讲者给出的理由是约束力。若直接在 Codex 或 Claude Code 之上叠加上层逻辑，常见做法会落到 `prompt`、`hook` 或类似扩展机制上。`prompt` 像给工人写的一张施工便签，能描述期望，难以保证每个状态转换都被严格记录；`hook` 像门口的检查点，能截获若干事件，覆盖不了一整套图状态、意图生成、任务分发、结果回写的协议语义。Cairn 需要的是外层可审计、可限定、可调度的框架，确保 worker 只能在图给出的任务语境中行动，并把结果沉淀回统一状态。

协议层的职责

这一段问答确认了 Cairn 的工程主张：Codex 与 Claude Code 是可替换的强执行器，外层协议负责定义状态、约束任务、派发 `intent`、回收 `fact`。真正被评审的是这层协议能否稳定地把大模型能力压进黑板图循环。

这里需要避免一个常见误解：外层协议的独立性并不来自对底层模型能力的不信任，它来自系统语义的归属问题。模型可以生成代码、读文件、执行探索、综合线索，但“哪一个事实已经被确认”“哪一个意图等待执行”“哪一次图变化触发新推理”这些状态管理职责需要稳定的数据结构承载。让模型在自然语言里自觉维护这些职责，风险会集中到提示词服从度上；用协议层维护这些职责，风险转移到协议设计、调度实现和成本控制上。

¹⁹ 视频画面时间区间：00:12:00–00:13:17。



图 19: 问答回到系统架构页后，画面确认 Codex 与 Claude Code 位于容器执行器层，外侧还有 Cairn Server 与 Dispatcher 协议边界。¹⁹

6.2 Token 与金钱成本：架构代价已经暴露

评委随后追问调用过程是否重复了模型交互，从而增加 token。讲者的回答很直接：确实会增加。原因有两层。第一，每次 Claude Code 一类执行器内部自带的上下文已经很长；第二，Cairn 的图上下文本身也会随着事实、意图、尝试记录和失败信息逐步膨胀。两种上下文叠加，就会把成本从单次调用成本推向系统运行成本。

字幕中保留下来的数量级很关键：整套架构在五天全程不断机运行的情况下，消耗约 10 亿 token，花费 7000 多人民币。这个数字不应被写成趣闻，因为它给出了架构可扩展性的第一条硬约束。若图越长、推理越频繁、worker 越多，成本增长会同来自上下文长度、调用次数和失败探索的回写。

成本是核心约束

成本属于核心约束。黑板图把失败也变成资产，但失败资产会占用上下文；外层协议提高了可控性，也增加了调度与重复交互开销。若未来任务规模更大，需要明确图压缩、事实摘要、冷热上下文分层和预算上限，否则“持续探索”很容易变成持续烧钱。

底层名称也需要按证据谨慎处理。问答前半段明确出现了 Codex 和 Claude Code；成本回答之后，讲者被问到“最后用的模型是什么”，字幕显示为 GPT 以及一处疑似 Claude Code 的 ASR 误写。由此可稳妥记录为：Cairn 的执行端涉及 Codex、Claude Code 等代码代理或执行器，模型侧提到了 GPT；具体产品拼写应由画面复核，不把 ASR 中的 CONDEX、COCODE、CROCODEOS 当成新组件。

模型与执行器

“模型”和“执行器”在这里要分开看。模型是产生推理、代码或计划的能力核心，例如 GPT；执行器是把任务包装成可运行工作流的外壳，例如 Codex 或 Claude Code。Cairn 的协议层调用的是执行器形态的 worker，worker 内部再依赖具体模型能力。

6.3 依赖关系：由图态势推断，不靠系统预设

第二位评委把问题推进到调度层：intent 的分发会形成任务图，任务之间可能存在依赖，系统如何处理这些依赖；执行时是否考虑并发，还是严格顺序运行。讲者承认任务确实有依赖，并用一个具体例子说明依赖如何出现：一个节点来自两个上游事实，一个事实是拿到一组密码，另一个事实是找到登录接口；AI 读取图后认为这两者可以组合，进而生成“用这组密码登录这个接口”的下游 intent。

这个例子很小，却把 Cairn 的依赖模型讲清楚了。依赖并非由人预先写成固定工作流，也不由系统硬编码成“先找密码、再找登录点、再登录”。依赖来自黑板图中事实节点之间的可组合性，由 reason 任务在读取全图后判断。两个上游 fact 的语义一旦能支撑一个新的动作，下游 intent 就出现了。

依赖是事实合流

依赖关系可以理解为“事实合流”。单个事实像一块拼图，未必足以触发下一步；多个事实在图中被推理任务读到后，可能共同支撑一条新的探索边。Cairn 的调度粒度因此落在图态势上，而非固定流程表上。

用符号写，这个关系可以粗略表示为：

$$f_{\text{pwd}} + f_{\text{login}} \xrightarrow{R(G)} e_{\text{try_login}}$$

- G ：当前黑板图，包含已经确认的事实和待执行的意图。
- f_{pwd} ：表示“已经拿到一组密码”的 fact。
- f_{login} ：表示“已经找到一个登录接口”的 fact。
- $R(G)$ ：读取全图并进行调度判断的 reason 过程。
- $e_{\text{try_login}}$ ：由上游事实共同支撑的新 intent 边，含义是尝试用该密码登录该接口。

6.4 串行推理与并行执行：调度边界在哪里

并发问题需要拆成两层看。讲者明确说，推理在 Cairn 中是串行的：只有当整张图发生变化时才触发推理，推理时会获取完整上下文。这样做的好处是每次 reason 都能看到全局态势，减少局部 worker 各自为政带来的冲突。代价同样明显：推理过程像单车道收费站，所有新事实带来的方向判断都要通过它。

执行层的边界不同。reason 读完整张图后，可以产出一条探索边，也可以产出多条探索边。简单题里可能只有一条边在跑；复杂图里，同一个节点可以衍生出两条甚至更多不同的边，这些边可以交给多个 worker 同时执行。并发发生在 $W(e_j)$ 这一层，即 worker 对具体探索边的执行，而非全图 reason 本身。

把这个调度关系写成一个伪形式化表达，就是：

$$\Delta G \Rightarrow R(G) \rightarrow \{e_1, e_2, \dots, e_n\}, \quad W(e_1) \parallel W(e_2) \parallel \dots \parallel W(e_n)$$

²⁰ 视频画面时间区间：00:14:33–00:14:58。

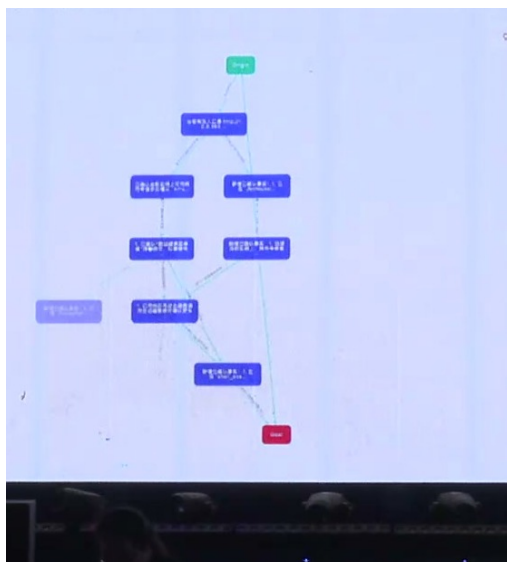


图 20: 复杂案例图中两条上游事实线汇合成下游动作, 适合作为“密码事实 + 登录接口事实生成登录 intent”的依赖关系证据。²⁰

- ΔG : 图发生变化, 例如新增 fact、intent 完成或执行结果回写。
- $R(G)$: 串行 reason 过程, 读取当前完整黑板图。
- $\{e_1, e_2, \dots, e_n\}$: 一次 reason 产生的一组探索边, n 可以为 1, 也可以大于 1。
- e_j : 第 j 条 intent 或探索边。
- $W(e_j)$: 某个 worker 执行第 j 条探索边。
- $\parallel$: 这些 worker 执行可并行发生, 具体并行度受执行器数量、资源预算和调度策略限制。

串行与并行的分层

这里的“串行”更像总调度室每次先看完整地图, 再下发若干工单; “并行”更像多个施工队同时处理不同工单。总调度室保证全局一致性, 施工队提供吞吐量。若地图更新太频繁, 总调度室会排队; 若工单太少, 施工队会闲置。

评委进一步指出一个风险: 渗透类任务常常需要同时尝试很多思路, 串行推理可能限制系统在同一时间只能沿一条路走。讲者的澄清是, 一次推理可以生成多条带探索性质的边, 再由 worker 并行分发。他还补充说, 这里的 **reason** 不是静态读图; 它会拿到完整上下文, 并可做轻量探索、查阅本地知识库或本地材料, 再生成多条候选探索边。这个细节解释了为什么一次串行推理仍可能输出多个方向。也就是说, Cairn 的并发路径是: 先由一个全局 **reason** 产出多个方向, 再把执行摊开; 当前问答描述的边界更窄: 一个全局 **reason** 先统一产出方向, 再由多个 worker 并行执行。

6.5 瓶颈与未解决问题

问答最后留下的是一组必须正视的瓶颈, 答案仍不完整。

²¹ 视频画面时间区间: 00:15:05–00:16:47。

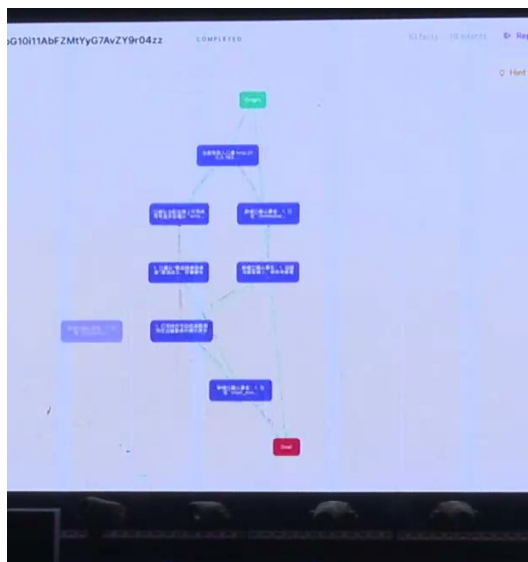


图 21: 复杂图展示一次 reason 后出现多条可探索分支的形态；串行 reason 与并行 worker 分发的关系仍建议配合正文公式解释。²¹

第一，推理串行会限制大规模并发。只要 reason 必须读取完整图并统一产出 intent，系统吞吐量就会受全局图大小、推理耗时和触发频率影响。这个设计换来了全局一致性，也把“下一步方向”的判断集中到了单一调度过程。

第二，上下文成本会随图增长上升。黑板图越忠实地保存探索痕迹，就越需要压缩、分层或裁剪。否则每一轮 reason 都会背着越来越重的历史包袱。用数学直觉看，若一次推理的上下文长度近似为 $|G|$ ，推理次数为 m ，总 token 成本至少会随 $m \cdot |G|$ 的量级增长；当 worker 内部上下文也很长时，常数项会非常高。

第三，依赖由 AI 推断带来灵活性，也带来可解释性压力。密码和登录接口的例子很自然，其他领域的依赖可能更隐蔽。系统需要记录“为什么这两个 fact 支撑这条 intent”，否则回溯失败路径时只能看到结果，看不到判断依据。

评审遗留风险

架构评审中的关键结论很朴素：Cairn 把 worker 的能力组织起来了，但尚未消除调度、成本和依赖解释的问题。赛场表现说明方向有信号，工程化复现仍需要更严格的预算控制、图摘要策略、依赖证据记录和并发压力测试。

6.6 本章小结

问答环节把 Cairn AI 从概念演讲推进到工程审查。这些追问共同说明，Cairn 的难点已经从单次解题能力推进到状态协议能否长期承载多执行器协作。

读者应带走的判断是：Cairn 的核心难点不只在“让模型会做渗透题”，还在“让多个强执行器服从同一张图”。这张图提供了统一上下文和任务边界，也制造了新的工程压力。协议、成本、依赖、并发四个问题如果处理不好，黑板图会从协作中枢变成昂贵瓶颈；处理得足够克制，它才可能成为通用问题求解的稳定骨架。

7 总结与延展：从渗透测试路径到通用求解骨架

这场分享的主线可以压缩成一条清晰链路：渗透测试被改写为状态空间里的有向搜索，Cairn 用黑板图承载这个搜索过程，worker 通过 `bootstrap`、`reason`、`explore` 三类任务读写这张图，外层协议再把 Codex、Claude Code 等强执行器约束在同一套状态语义里。讲者的结尾和评委问答共同提醒读者：这套设计有赛场信号，也有明确工程代价。

7.1 讲者收束：成绩、风险与 Less is More

讲者在结尾给出了几项具体事实。队伍线上第四名，开局配置失误导致前期抢分受损，最终成为全场唯一 AK 所有题目的队伍；系统在赛前没有完整测试，比赛当天凌晨四点才第一次调通。这些信息放在一起，构成了一个克制的判断：Cairn 的方向在真实竞赛压力下跑出了强信号，同时它当时仍带着明显的原型风险。

Less is More 在这里也应按工程命题理解。讲者没有把复杂性推给更多安全工具、更多 MCP、更多 RAG、更多预定义 agent 角色，而是追问渗透测试的最小状态表达。这个判断的锋利处在于：系统层数减少之后，每一层都必须解释自己的必要性。黑板图负责保存公共状态，任务循环负责生成下一步行动，worker 负责执行和回写；其余能力可以存在，但需要证明会增强搜索结构，不能只复制人工流程。

讲者最后把 Less is More 压成一句话：系统没有教 agent 渗透测试，只给了它们一块黑板、一个目标和一堆工具，剩下的路径由图中事实、意图与执行结果的循环自行涌现。这句话把整场分享重新拉回核心机制：图保存状态，目标提供方向，工具提供动作能力，路径从多轮回写中长出来。

结尾的真实含义

赛场唯一 AK 说明图中心搜索有有效信号；凌晨四点首次调通说明系统成熟度仍需验证。Less is More 的价值在于逼迫设计者回到问题本体：哪些状态必须保存，哪些动作必须派发，哪些结果必须回写。

7.2 四个核心判断

第一，问题被改写为图上的有向搜索。渗透测试、漏洞挖掘、数学证明这类任务都可以拥有明确起点、明确达成标准和未知中间路径。若把求解过程写成

$$v_0 \rightsquigarrow V_T$$

v_0 是起点状态， V_T 是目标状态集合， $\rightsquigarrow$ 表示中间路径需要通过探索逐步发现。这个表达不会替代领域知识，它只给系统一个承载领域知识的骨架。

第二，黑板图把上下文、意图和执行结果统一到共享状态。单轮 worker 的发现若停留在私有上下文中，后续 worker 无法稳定接住；写入 `fact`、`intent`、`hint` 之后，系统才有连续推进的公共记忆。

第三，worker 的能力来自任务与图态势的结合。worker 没有固定身份，本轮可以 `reason`，下一轮可以 `explore`。任务来自图的当前态势，态势来自前一轮写回。这个闭环让分工在运行时生成，避免过早把问题切成固定角色。

第四，外层协议提供强约束，同时带来成本与调度压力。问答中提到五天不断机、约 10 亿 token、7000 多人民币，这组数字属于系统可扩展性的硬证据。协议层越完整，状态越可审计；图上下文越长，推理与执行成本越高。

不能淡化的代价

Cairn 把多个强执行器组织到同一张图里，但代价集中在三处：推理串行可能形成瓶颈，图上下文增长会推高 token 成本，依赖关系由 AI 推断会带来解释压力。后续工程化必须处理预算、摘要、回滚、并发和审计。

7.3 什么时候适合这种架构

Cairn 的结构对某些问题特别有吸引力。目标必须足够明确，例如拿到全部 flag、证明某个命题、确认某类风险；中间路径必须足够未知，固定流程很难覆盖；探索结果必须能回写成结构化事实；失败信息必须有价值，可以减少重复尝试或改变后续推理；任务必须能从图态势中生成，而非完全依赖人工逐步拆单。

用更直观的类比说，它适合“边走边画地图”的问题。若地图在开始时已经完整，固定流程更便宜；若目标模糊到无法验收，图会积累大量无法判定价值的痕迹；若执行结果不能结构化回写，黑板图会退化成长日志。Cairn 的优势恰好出现在目标清楚、路径未知、反馈可沉淀的区域。

读者可迁移的判断框架

评估一个任务能否使用黑板图架构，可以先问五个问题：起点是什么，终点如何验收，新事实怎样写回，失败怎样转化为后续信息，下一步任务能否由当前图态势生成。五个问题回答不清，系统很可能会在外层调度上消耗过多成本。

7.4 延展：从安全竞赛走向通用问题求解

这场分享最有价值的地方，在于它没有把 AI agent 只当作“更聪明的工具调用器”。Cairn 试图回答一个更底层的问题：当任务需要多轮探索、多次失败、多条候选路径和多个执行器时，系统怎样保存共同进展。它给出的答案是黑板图：把问题进展沉淀到共享图里，让下一步行动由图的当前态势生成。

这条思路可以延展到其他领域，但延展必须谨慎。数学证明、代码调试、研究综述、复杂运维都可能符合“起点明确、目标可验收、中间路径未知、反馈可回写”的结构。它们与渗透测试的差异也很真实：证明需要形式严谨，调试需要可复现证据，研究需要来源可信，运维需要变更安全。Cairn 的骨架可以迁移，领域约束需要重新写入图语义、任务类型和验证规则。

最终，读者可以这样记住 Cairn：它是一种图中心的搜索引擎。它不把智能藏在某个万能 agent 里，也不把流程钉死在人工角色表里；它把已知事实、下一步意图、外部提示和执行结果统一到一张可持续更新的黑板图中。只要这张图能保持一致、成本能被控制、依赖能被解释，worker 群体就有机会从简单的读图、行动、回写循环中长出具体的问题求解路径。

7.5 本章小结

整篇笔记的主旨落在四个词上：状态、图、任务、边界。状态空间解释渗透测试为何可被建模，黑板图保存公共进展，三类任务推动图不断生长，协议边界约束强执行器并暴露成本。Cairn 的贡献在于把“会做题的模型”推进到“能沉淀问题进展的系统”。这也是后续评价它的标准：看它能否在更多任务中保持图一致性、控制成本、解释依赖，并让失败真正变成下一步搜索的材料。